

DATASTRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	B.Manaswini
ROLL NO	AV.SC.U4CSE25102
YEAR/SEM/SECTION	2ndYEAR/ SEM-3/CSE-B
CREDITS	05

SUBMITTED TO	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR(SL Grade)
DEPARTMENT	SCHOOL OF COMPUTING

Index

S.no	Title	Page.no	Remark
1.	**A)** A rocket launch system displays a countdown before launch. Instead of using loop, implement the countdown using recursion. Write a recursive program to display a countdown from n to 1 followed by launch. **B)** Bank calculates the growth of an investment using compound intrest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate pn(power), where P=principal growth factor, n=no.of years. **C)** Write a recursive progrsm to print the first n terms of Fibonacci series **D)** An hr department stores employee ids in a list develop a recursive program to search for a given employee id. Write a recursive function to search for an employee id in the list. **E)** In a logistics the no.of possible ways to arrange n different parcels calculate using the factorial (n!). develop a recursive program to calculate the factorial of given number.	6–12	
2.	**A)** write a python program to perform linear search on the given input **B)** Write a python program to perform binary search on the given	13–16	

	input. Verify by taking sorted array and unsorted array as a list.		
3.	A) write a python program to implement bubble sort, where the elements are taken from user. B) write a python program to implement selection sort, where the elements are taken from user. C) write a python program to implement insertion sort, where the elements are taken from user.	17–21	
4.	A) write a python program to implement merge sort, where the elements are taken from user. B) write a python program to implement quick sort, where the elements are taken from user.	22–25	
5.	A)Write a python program to implement a single linked list with thebelow operations: A.create linked list B.insert at beginning C.Insert at end D.Insert at index E.Delete by value F.Delete first node G.Delete last node H.Count number of nodes I.Display/traverse J.exit B)Write a python program to implement a double linked list with the below operations: A.create linked list B.insert at beginning C.Insert at end D.Insert at index E.Delete by value	26–59	

	F.Delete first node G.Delete last node H.Count number of nodes I.Display/traverse J.exit C) Write a python program to implement a circular linked list with the below operations: A.create linked list B.insert at beginning C.Insert at end D.Insert at index E.Delete by value F.Delete first node G.Delete last node H.Count number of nodes I.Display/traverse J.exit		
6.	A) Write a python program to implement stack operations such as push, pop, peek, display using an array. B) Write a python program to implement stack operations such as push, pop, peek, display using linked list.	60–67	
7.	A) Write a python program to implement queue and its operations such as enqueue, dequeue, peek and display with a menu driven approach using an array. B) Write a python program to implement queue and its operations such as enqueue, dequeue, peek and display with a	68–75	

	menu driven approach using linked list.		
8.	a) Write a python program to implement a binary tree with its operations using array. b) Write a python program to implement a binary tree with its operations using linked list.	76–81	
9.	a) Write a python program to implement a binary search tree with its operations and traversals.	82–88	
10.	write a python program to implement an binary AVL tree with its operations	89–93	

LAB-1

1.AIM:A rocket launch system displaysa countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by “launch”.

PROGRAM:

```
def launch(n):
 for i in range(n,-1,-1):
 if i==0:
 print("launch")
 else:
 print(i)
n=int (input ("Enter countdown number:"))
launch(n)
```

OUTPUT:

```
Enter "help" below or click "Help" above for more information.

============== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lab1a.py ==========
Enter countdown number:9
9
8
7
6
5
4
3
2
```

EXPLANATION:

This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop

ERROR TABLE:

S.no.	Errors	Rectification
1.	Indentation Error	Use 3 **spaces** for each level of indentation.
2.	Type Error	Convert the input to an integer using n = int (input ("Enter no. of seconds to launch: ")).

2.AIM: The banks calculate the growth of an investment using compound interest; the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n(power), where p=principal growth factor, n= number of years.

PROGRAM:

```
def power (p, n):
 if n == 0:
 return 1
 else:
 return p * power (p, n - 1)
 p = float (input ("Enter principal/base value: "))
n = int (input ("Enter number of years/powers: "))
result = power (p, n)
print("Power =", result)
```

OUTPUT:

```
============== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lab1b.py ==============
Enter principal/base value: 321
Enter number of years/powers: 3
Power = 33076161.0
```

EXPLANATION:

- The function of power (p, n) calculates p^n using recursion by multiplying p with the result of power (p, n-1).
- The recursion stops when n becomes 0, returning 1, which is the base case.
- The final result is displayed after all recursive calls are completed.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Type Error	Convert the input to float or int using float(input()) or int(input()).
2.	Incorrect output if the recursive call is written as power(p, n + 1) instead of power(p, n - 1).	Change the recursive call to power(p, n - 1) so that n decreases toward the base case.

3.AIM: An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search (list, key, index):
 if index >= len(list):
 return False
 if list[index] == key:
 return True
 return search (lst, key, index + 1)
 employees = [101, 102, 103, 104, 105]
 key = int (input ("Enter Employee ID to search: "))
```

```
if search (employees, key, 0):
print (f"Employee ID found at position {key}")
else:
print ("Employee ID Not Found")
```

OUTPUT:

```
============== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lab1c.py ==============
Enter Employee ID to search: 102
Employee ID found at position 1
```

EXPLANATION:

- The recursive function checks each employee ID in the list one by one, starting from the first element.
- If the required ID is found, it returns True; otherwise, it continues searching recursively until the end of the list.
- If the entire list is searched without finding the ID, the function returns False.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Name Error if the list name is misspelled (e.g., employee instead of employees).	Use the correct variable name consistently throughout the program.
2.	Syntax Error due to a missing colon (:) after an if statement or function definition.	Add the missing colon (:) after if, else, or def.

4.AIM: In a logistics company the number of possible ways to arrange n different parcels are calculated using the factorial formula(n!). develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def factorial(n):
 if n == 0 or n == 1:
 return 1
 else:
 return n * factorial (n - 1)
num = int (input ("Enter a number: "))
if num < 0:
 print ("Factorial does not exist for negative numbers.")
else:
 print ("Factorial =", factorial(num))
```

OUTPUT:

```
============== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lab1d.py ==============
Enter a number: 2
Factorial = 2
```

EXPLANATION:

- The function factorial(n) computes the factorial by multiplying the current number with the factorial of the previous number.
- The recursion ends when n becomes 0 or 1, where the factorial value is 1.
- The program also checks for negative inputs since factorial is defined only for non-negative integers.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError if the statements inside the function are not properly indented.	Maintain proper indentation (usually 4 spaces) inside the function.
2.	TypeError if a decimal number (e.g., 5.5) is entered for factorial calculation.	Enter only a whole number or convert the input using int().

5.AIM: Write a recursive python program to print the first n terms of the Fibonacci series.

PROGRAM:

```
def fibonacci(n):
 if n == 0:
 return 0
 elif n == 1:
 return 1
 else:
 return fibonacci(n - 1) + fibonacci(n - 2)
n = int (input ("Enter the number of terms: "))
print ("Fibonacci Series:")
for i in range(n):
 print (fibonacci(i), end=" ")
```

OUTPUT:

```
============== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lab1e.py ======
Enter the number of terms: 5
Fibonacci Series:
0 1 1 2 3
```

EXPLANATION:

- The function Fibonacci(n) returns the nth Fibonacci number by recursively adding the previous two Fibonacci numbers.
- The base cases return 0 for the first term and 1 for the second term.
- A for loop calls the recursive function repeatedly to print the first n terms of the Fibonacci series.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Recursion Error if the base cases (n == 0 and n == 1) are not included.	Add both base conditions before the recursive call.
2.	Value Error if the user enters text instead of an integer.	Enter a valid integer for the number of terms.

LAB-2

1.AIM: Writeaprogramtoperformlinearsearch on a given input.

PROGRAM:

```
def linear_search(arr,key):
 for i in range(len(arr)):
 if arr[i]==key:
 return i
 return -1
 n = int(input("Enter no. of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
 arr.append(int(input()))
key=int(input("Enter element to search:"))
result=linear_search(arr,key)
if result!=-1:
 print("Element found at index",result)
else:
 print("Element not found.")
```

OUTPUT:

```
============== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lab2a.py ==============
Enter no. of elements: 3
Enter the elements:
10
20
60
Enter element to search: 60
Element found at index 2
```

EXPLANATION:

The linear_search()function checks each element of the list one by one and compares it with the given key.

- If the key is found, its index is returned; otherwise, the function returns -1.
- The program takes the array and search element as input and displays whether the element is found.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements and array elements.
2.	TypeError may occur if len(arr) is used on a variable that is not a list.	Make sure arr is initialized as a list using arr = [] and elements are appended to it.

2.AIM: Write aPython program to perform binary search on the given input. Verify by taking sortedarray and unsorted array as input.

PROGRAM:

```
def binary_search(arr,key):
 low=0
 high=len(arr)-1
 while low<=high:
 mid=(low+high)//2
 if arr[mid]==key:
 return mid
 elif arr[mid]<key:
 low=mid+1
 else:
 high=mid-1
 return -1
 n = int(input("Enter no. of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
 arr.append(int(input()))
 if arr==sorted(arr):
 print("\n The input list is already sorted.")
else:
print("\n The input list is not sorted.")
```

```
print("Sorting the list...")

 arr.sort()

print("Sorted list.",arr)

key=int(input("Enter element to search:"))

result=binary_search(arr,key)

if result!=-1:

 print("Element found at index",result)

else:

print("Element not found.")
```

OUTPUT:

```
============== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lab2b.py ========
Enter no. of elements: 2
Enter the elements:
10
20

The input list is already sorted.
Sorted list: [10, 20]
Enter element to search: 20
Element found at index 1
```

EXPLANATION:

- The binary_search() function repeatedly divides the sorted list into two halves and compares the middle element with the search key.
- If the middle element is smaller than the key, the search continues in the right half; otherwise, it searches the left half.
- The second program verifies whether the input is sorted and sorts it using arr.sort() if necessary before performing binary search.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndexError can occur if mid is calculated or the low/high boundaries are modified incorrectly.	Initialize low = 0, high = len(arr) - 1 and update them as low = mid +1 or high = mid - 1.

Lab-3

1.AIM: write programs for the Bubble sort

PROGRAM:

```
#bubble sort
def bubble_sort(arr):
    n = len(arr)
    for i in range (n-1):
        for j in range(n-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1] = arr[j+1],arr[j]
    return arr
#taking input
n = int(input("Enter number of elements: "))
arr=[]
print("Enter elements: ")
for i in range(n):
    arr.append(int(input()))
bubble_sort(arr)
print("sorted array: ")
for element in arr:
print(element, end=" ")
```

Output:

```
============== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lab3a.py =======
Enter number of elements: 4
Enter elements:
10
20
98
65
Sorted array:
10 20 65 98
```

Explanation:

Bubble sortcompares two adjacent elements. If the left element is greater than the right element, they are swapped. After each pass, the largest remaining element moves to the end.

Error table:

S.no	Error	Correction
1.	Used wrong loop range	Use for j in range(n-i-1):
2.	Swap condition is wrong	Use if arr[j] > arr[j+1]:

2.AIM: writeprograms for the Insertion sort

PROGRAM:

```
# Insertion Sort


def insertion_sort(arr):

    n = len(arr)

    for i in range(1, n):

        key = arr[i]

        j = i - 1

        while j >= 0 and arr[j] > key:

            arr[j + 1] = arr[j]
```

j1 8= | Pja -ge 1

```
        arr[j + 1] = key
    return arr
# Taking input
n = int(input("Enter number of elements: "))
arr = []
print("Enter elements: ")
for i in range(n):
    arr.append(int(input()))
insertion_sort(arr)
print("Sorted array: ")
for element in arr:
    print(element, end=" ") same for this
```

Output:

```
============== RESTART: D:/manaswinii-25102/LAB PROGRA
Enter number of elements: 3
Enter elements:
11
76
50
Sorted array:
11 50 76
```

Explanation:

Insertion sort builds the sorted array one element at a time.

It takes one element called the **key**, compares it with the elements before it, shifts larger elements to the right, and inserts the key in the correct position.

Error table:

S.no	Error	Correction
1.	key is not stored properly	Use key = arr[i]
2.	arr[j+1] = key placed inside while loop	Place it after the while loop

3.AIM: writeprograms for the Bubble sort

PROGRAM:

```
# Selection Sort
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
 return arr
# Taking input
n = int(input("Enter number of elements: "))
arr = []
print("Enter elements: ")
for i in range(n):
```

```
    arr.append(int(input()))

selection_sort(arr)

print("Sorted array: ")

for element in arr:

print(element, end=" ")
```

Explanation:

Selection sort repeatedly finds the **smallest element** from the unsorted part of the array and places it in its correct position.

Error table:

S.no	Error	Correction
1.	min_index is not initialized	Use min_index = i
2.	Swap is placed inside inner loop	Place swap after the inner loop

LAB-4

1.Aim: Write a python program to implement merge sort where the input elements are taken from the user

PROGRAM:

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
```

```
    while j < len(right):
        arr[k] = right[j]
        j += 1
        k += 1
# Input from user
arr = list(map(int, input("Enter elements separated by spaces: ").split()))
merge_sort(arr)
print("Sorted array:", arr)
```

Output:

```
========== RESTART: D:/DSA Programs/programs/Week4.py ========
manaswini-cse25102
Enter elements separated by spaces: 23 75 32 11 8 45 08 53
Sorted array: [8, 8, 11, 23, 32, 45, 53, 75]
```

Explanation:

Merge Sort is a **divide-and-conquer sorting algorithm**.It works in three main steps:

1. Divide – Divide the array into two halves.
2. Conquer – Recursively sort both halves.
3. Merge – Merge the two sorted halves into one sorted array.

Error table:

S.no	Error	Correction
1.	Wrong base condition	Use if len(arr) > 1 for splitting
2.	Wrong midpoint	Use mid = len(arr)//2
3.	Forgetting remaining elements	Append remaining elements

2.Aim: Write a python program to implement quick sort where the input elements are taken from the user

Program:

```
defquick_sort(arr, low, high):


    if low < high:

        pivot = arr[high]

        i = low - 1

        for j in range(low, high):

            if arr[j] < pivot:

                i += 1

                arr[i], arr[j] = arr[j], arr[i]

        arr[i + 1], arr[high] = arr[high], arr[i + 1]

        p = i + 1

        quick_sort(arr, low, p - 1)

        quick_sort(arr, p + 1, high)

# Input from user

arr = list(map(int, input("Enter numbers to sort: ").split()))

quick_sort(arr, 0, len(arr) - 1)

print("Sorted array:", arr)
```

Output:

```
= RESTART: C:/Users/CSE Lab1/Desktop/25102/DSA-25102/we
manaswini-cse25102
Enter numbers to sort: 12 34 16 57 85
Sorted array:
[12, 16, 34, 57, 85]
```

~~Explanation:~~

Quick Sort is also a **divide-and-conquer sorting algorithm**.

Its main idea is to select an element called a **pivot** and arrange the array so that:

Elements smaller than pivot | Pivot | Elements greater than pivot

Then Quick Sort is recursively applied to the left and right portions.

~~Error table:~~

S.No.	Error	Correction
1	Incorrect Python syntax, missing : or brackets	Check syntax, brackets and colons
2	Incorrect low/high boundaries are passed	Use low, pivot-1 and pivot+1, high
3	Function keeps calling itself	Add if low < high:

LAB-5

1.Aim: Write a Python program to implement a singly linked list with the below operations:

1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at a specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

Program:

```
class Node:

    def__init__(self, data):

        self.data = data

        self.next = None

class SinglyLinkedList:

    def __init__(self):

        self.head = None

    #1.Create linked list

    def create(self):

        n=int(input("Enter number of nodes: "))

        foriin range(n):

            data = int(input(f"Enter data for node {i + 1}: "))

  new_node = Node(data)

            ifself.head is None:

                self.head = new_node
```

```
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next= new_node
    print("Linkedlist created successfully.")
# 2. Insert at beginning
def insert_beginning(self):
    data=int(input("Enter data: "))
    new_node = Node(data)
    new_node.next= self.head
    self.head = new_node
    print("Nodeinserted at beginning.")
# 3. Insert at end
def insert_end(self):
    data=int(input("Enter data: "))
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
    else:
        temp = self.head
```

```
while temp.next:
            temp = temp.next
        temp.next = new_node
    print("Node inserted at end.")
#4.Insertat specific index
def insert_index(self):
    index=int(input("Enter index: "))
    data=int(input("Enter data: "))
    new_node = Node(data)
    ifindex== 0:
        new_node.next = self.head
        self.head = new_node
        print("Node inserted.")
        return
    temp = self.head
    foriinrange(index - 1):
        iftemp is None:
            print("Invalid index.")
            return
        temp= temp.next
    iftempis None:
        print("Invalid index.")
        return
```

```
 new_node.next = temp.next
    temp.next = new_node
    print("Node inserted.")
#5.Delete by value
def delete_value(self):
    value = int(input("Enter value to delete: "))
    ifself.head is None:
        print("List is empty.")
        return
    ifself.head.data == value:
        self.head = self.head.next
        print("Node deleted.")
        return
    temp = self.head
    while temp.next and temp.next.data != value:
        temp = temp.next
    iftemp.next is None:
        print("Value not found.")
    else:
        temp.next = temp.next.next
        print("Node deleted.")
#6.Delete first node
def delete_first(self):
```

```
  ifself.head is None:
        print("List is empty.")
else:
        self.head = self.head.next
        print("First node deleted.")
 #7.Delete last node
 def delete_last(self):
    ifself.head is None:
        print("List is empty.")
        return
    ifself.head.next is None:
        self.head = None
        print("Last node deleted.")
        return
    temp = self.head
    while temp.next.next:
        temp = temp.next
    temp.next = None
    print("Last node deleted.")
 #8.Count nodes
 def count(self):
    count = 0
    temp = self.head
    while temp:
```

```
        count += 1
            temp = temp.next
        print("Number of nodes:", count)
    # 9. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty.")
            return
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")
# Main program
sll = SinglyLinkedList()
while True:
    print("\n--- SINGLY LINKED LIST ---")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
```

```
print("8. Count number of nodes")
print("9. Display / Traverse")
print("10. Exit")
choice = int(input("Enter your choice: "))
ifchoice == 1:
    sll.create()
elifchoice == 2:
    sll.insert_beginning()
elifchoice == 3:
    sll.insert_end()
elifchoice == 4:
    sll.insert_index()
elifchoice == 5:
    sll.delete_value()
elifchoice == 6:
    sll.delete_first()
elifchoice == 7:
    sll.delete_last()
elifchoice == 8:
    sll.count()
elifchoice == 9:
    sll.display()
elifchoice == 10:
    print("Program exited.")
```

```
        break
    else:
        print("Invalid choice.")
```

OUTPUT:

```
================== RESTART: D:/DSA Programs/programs/Week5a.py ==================

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter data: 14
Enter data: 23
Enter data: 56
Linked List created successfully

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 2
Enter data: 27

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 3
Enter data: 4
```


```
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 3
Enter data: 4
```


```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
```

```
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 5

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 6

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 7
Enter index: 6
Index out of bounds!

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 8
Number of nodes: 4

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
```

```
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
14 -> 23 -> 56 -> 4 -> None

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 10
```

Explanation:

SinglyLinkedList – Explanation

A **singlylinkedlist** isalineardatastructure in which elements are stored in parts:

1. **Data** – stores the value.
2. **Next** – stores the address/reference of the next node.

- self.data stores the node value.
- self.next stores the reference to the next node.
- Initially, next = None because the node is not connected to another node.

Error table:

S.No	Mistake	Correction
1	`Node` class is not defined before using it	Define the Node class first
2	`temp` is not updated inside the loop	Add `temp = temp.next`
3	`head` is not updated	Use `self.head = new_node`
4	User enters an index outside the list range	Validate the index before insertion
5	The given value does not exist	Search the entire list and display "Value not found"
6	head is None and operation is performed directly	Check `if self.head is None`

2.Aim: Write a Python program to implement a Double linked list with the above operations

Program:

```
class Node:
 def __init__(self, data):
    self.data  =  data
    self.prev  =  None
    self.next = None
```

```
class DoublyLinkedList:
    def __init__(self):
        self.head = None
    # 1. Create linked list
    def create(self):
        n = int(input("Enter number of nodes: "))
        for i in range(n):
            data = int(input(f"Enter data for node {i + 1}: "))
            new_node = Node(data)
            if self.head is None:
                self.head = new_node
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next
                temp.next = new_node
                new_node.prev = temp
        print("Doubly linked list created successfully.")
    # 2. Insert at beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)
```

```
    new_node = Node(data)
    ifself.headisnot None:
        new_node.next = self.head
        self.head.prev = new_node
    self.head = new_node
    print("Nodeinserted at beginning.")
# 3. Insert at end
def insert_end(self):
    data=int(input("Enter data: "))
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
    else:
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node
        new_node.prev = temp
    print("Nodeinserted at end.")
#4.Insertatspecific index
def insert_index(self):
    index=int(input("Enter index: "))
    data=int(input("Enter data: "))
```

```
new_node = Node(data)
ifindex == 0:
    if self.head:
        new_node.next = self.head
        self.head.prev = new_node
    self.head = new_node
    print("Node inserted.")
    return
temp= self.head
foriin range(index - 1):
    iftemp is None:
        print("Invalid index.")
        return
    temp = temp.next
iftemp is None:
    print("Invalid index.")
    return
new_node.next = temp.next
new_node.prev = temp
if temp.next:
    temp.next.prev = new_node
temp.next = new_node
print("Node inserted.")
```

```
# 5. Delete by value
    def delete_value(self):
        value = int(input("Enter value to delete: "))
        temp = self.head
        while temp and temp.data != value:
            temp = temp.next
        if temp is None:
            print("Value not found.")
            return
        if temp.prev:
            temp.prev.next = temp.next
        else:
            self.head = temp.next
        if temp.next:
            temp.next.prev = temp.prev
        print("Node deleted.")
    # 6. Delete first node
    def delete_first(self):
        if self.head is None:
            print("List is empty.")
            return
        self.head = self.head.next
        if self.head:
            self.head.prev = None
```

```
print("First node deleted.")
#7.Delete last node
def delete_last(self):
    ifself.head is None:
        print("List is empty.")
        return
    temp = self.head
    while temp.next:
        temp = temp.next
    if temp.prev:
        temp.prev.next = None
    else:
        self.head = None
    print("Last node deleted.")
#8.Count nodes
def count(self):
    count = 0
    temp = self.head
    while temp:
        count += 1
        temp = temp.next
    print("Number of nodes:", count)
#9.Display / Traverse
def display(self):
```

```
        if self.head is None:
            print("List is empty.")
            return
        temp = self.head
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")
# Main program
dll = DoublyLinkedList()
while True:
    print("\n--- DOUBLY LINKED LIST ---")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
 print("8. Count number of nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))
```

```
ifchoice == 1:
    dll.create()
elifchoice == 2:
    dll.insert_beginning()
elifchoice == 3:
    dll.insert_end()
elifchoice == 4:
    dll.insert_index()
elifchoice == 5:
    dll.delete_value()
elifchoice == 6:
    dll.delete_first()
elifchoice == 7:
    dll.delete_last()
elifchoice == 8:
    dll.count()
elifchoice == 9:
    dll.display()
elifchoice == 10:
    print("Program exited.")
    break
else:
    print("Invalid choice.")
```

Output:

```
================== RESTART: D:/DSA Programs/programs/Week5b.py ==================

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter number of nodes: 4
Enter data: 6
Enter data: 8
Enter data: 8
Enter data: 3
Linked List created successfully

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 2
Enter data: 5

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 3
Enter data: 65

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 4
Enter index: 3
Enter data: 90

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
```

```
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 5

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 6

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 7
Enter index: 4

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 8
Number of nodes: 4

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
```

```
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
6 -> 8 -> 90 -> 8 -> None

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 10
```

Explanation:

A Doubly Linked List is a linear data structure in which each node contains **three parts**:

1. prev – reference to the previous node
2. data – stores the value
3. next – reference to the next node

head points to the first node.

The first node's prev is None.

The last node's next is None.

Each node can be traversed **forward and backward**.

S.No	Error	Correction
1	Trying to access prev of a nonexistent node	Use an appropriate if condition
2	prev links are not updated	Correctly update prev references
3	head becomes None	Check whether the list becomes empty
4	Second-last node's next is not changed	Set temp.prev.next = None
5	Index is outside the list	Validate the index

3..Aim: Write a Python program to implement a Circular linked list with the above operations

Program:

```
class Node:

  def __init__(self, data):

     self.data = data

     self.next = None

class CircularLinkedList:

    def __init__(self):

         self.head = None
```

```
# 1. Create linked list
 def create(self):
    n=int(input("Enter number of nodes: "))
    for i in range(n):
        data=int(input(f"Enter data for node {i + 1}: "))
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head
            whiletemp.next != self.head:
                temp = temp.next
            temp.next = new_node
            new_node.next = self.head
    print("Circularlinked list created successfully.")
 # 2. Insert at beginning
 def insert_beginning(self):
    data=int(input("Enter data: "))
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
        new_node.next= self.head
```

```
    else:
        temp = self.head
        while temp.next != self.head:
            temp = temp.next
        new_node.next = self.head
        temp.next = new_node
        self.head = new_node
    print("Node inserted at beginning.")
#3.Insert at end
def insert_end(self):
    data= int(input("Enter data: "))
    new_node = Node(data)
    ifself.head is None:
        self.head = new_node
        new_node.next = self.head
    else:
        temp = self.head
        while temp.next != self.head:
            temp = temp.next
        temp.next = new_node
        new_node.next = self.head

    print("Node inserted at end.")
```

```
#4.Insertat specific index
def insert_index(self):
    index=int(input("Enter index: "))
    data=int(input("Enter data: "))
    new_node = Node(data)
    ifindex== 0:
        ifself.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            new_node.next = self.head
            temp.next = new_node
            self.head = new_node
        print("Node inserted.")
        return
    ifself.head is None:
        print("Invalid index.")
        return
    temp = self.head
    foriinrange(index - 1):
        temp= temp.next
```

```
# 5. Delete by value
def delete_value(self):
    value=int(input("Enter value to delete: "))
    if self.head is None:
        print("List is empty.")
        return
    current = self.head
    previous = None
    while True:
        ifcurrent.data== value:
            break
        previous = current
        current = current.next
        if current == self.head:
            print("Valuenot found.")
            return
    # Only one node
    ifcurrent==self.head and current.next == self.head:
        self.head = None
    # Delete head
    elif current == self.head:
        last = self.head
        whilelast.next!= self.head:
            last = last.next
```

```
        self.head = self.head.next
        last.next = self.head
    #Deleteother node
    else:
        previous.next = current.next
    print("Node deleted.")
#6.Deletefirst node
def delete_first(self):
    ifself.head is None:
        print("List is empty.")
        return
    #Onlyone node
    ifself.head.next == self.head:
        self.head = None
    else:
        last = self.head
        whilelast.next != self.head:
            last= last.next
        self.head = self.head.next
        last.next = self.head
    print("First node deleted.")
#7.Deletelast node
def delete_last(self):
    ifself.head is None:
```

```
        print("List is empty.")
        return
    # Only one node
    if self.head.next == self.head:
        self.head = None
        print("Last node deleted.")
        return
    temp = self.head
    whiletemp.next.next!= self.head:
        temp = temp.next
    temp.next = self.head
    print("Last node deleted.")
# 8. Count nodes
def count(self):
    if self.head is None:
        print("Numberofnodes: 0")
        return
    count = 0
    temp = self.head
    while True:
        count += 1
        temp = temp.next
        if temp == self.head:
            break
    print("Numberofnodes:", count)
```

```
    # 9. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty.")
            return
        temp = self.head
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.head:
                break
        print("(back to head)")
# Main program
cll = CircularLinkedList()
while True:
    print("\n--- CIRCULAR LINKED LIST ---")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
 print("8. Count number of nodes")
```

```
print("9. Display / Traverse")
print("10. Exit")
choice = int(input("Enter your choice: "))
ifchoice == 1:
    cll.create()
elifchoice == 2:
    cll.insert_beginning()
elifchoice == 3:
    cll.insert_end()
elifchoice == 4:
    cll.insert_index()
elifchoice == 5:
    cll.delete_value()
elifchoice == 6:
    cll.delete_first()
elifchoice == 7:
    cll.delete_last()
elifchoice == 8:
    cll.count()
elifchoice == 9:
    cll.display()
elifchoice == 10:
    print("Program exited.")
    break
else:
    print("Invalid choice.")
```

Output:

```
================== RESTART: D:/DSA Programs/programs/Week5c.py ==========

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter data: 21
Enter data: 78
Enter data: 11
Linked List created successfully

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 2
Enter data: 4

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 3
Enter data: 8

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 4
Enter index: 2
Enter data: 54

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
```

```
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 5

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 6

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 7
Enter index: 2

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 8
Number of nodes: 3

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
```

```
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
21 -> 54 -> 11 -> (head)

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 10
```

Explanation:

A Circular Linked List is a linked list in which the **last node points back to the first node** instead of pointing to None.

New node points to the old head.

Last node points to the new head.

Error table:

S.No	Error	Correction
1	Infinite loop during traversal	Stop when `temp == head`
2	Circular connection is lost	Set last node's `next` to `head`
3	Insert at beginning breaks the circle	Update `last.next = new_node`
4	Count is incorrect	Stop when pointer reaches `head`
5	Invalid index	Check index before insertion

LAB-6

1.Aim: Write a Python programtoimplementstackoperations such as Push, Pop, Peek, Display using an array

Program:

```
class Stack:
    def __init__(self, capacity):
        self.capacity = capacity
        #Initialize the array with None and track the top index
        self.stack = [None] * capacity
        self.top = -1

    #1.Push Operation
    def push(self):
        ifself.top == self.capacity - 1:
            print("Stack Overflow! Cannot push element.")
            return

        data = int(input("Enter data to push: "))
        self.top += 1
        self.stack[self.top] = data
        print(f"{data} pushed onto the stack.")

    #2.Pop Operation
    def pop(self):
        ifself.top == -1:
            print("Stack Underflow! Stack is empty.")
            return

        popped_element = self.stack[self.top]
        self.stack[self.top] = None # Clear the slot
        self.top -= 1
        print(f"Popped element: {popped_element}")

    #3.Peek Operation
    def peek(self):
        ifself.top == -1:
            print("Stack is empty!")
            return

        print(f"Top element is: {self.stack[self.top]}")
```

```
    # 4. Display Operation
    def display(self):
        if self.top == -1:
            print("Stack is empty!")
            return

        print("Stack elements (Top to Bottom):")
        # Loop backwards from top down to 0
        for i in range(self.top, -1, -1):
            print(f"| {self.stack[i]} |")
        print("-----")


def main():
    try:
        size= int(input("Enter the size of the stack: "))
    except ValueError:
        print("Please enter a valid size integer. Defaulting size to 5.")
        size= 5

    stack_obj = Stack(size)

    while True:
        print("\n----- STACK OPERATIONS -----")
        print("1. Push")
        print("2. Pop")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")

        try:
            choice = int(input("Enter your choice: "))
        except ValueError:
            print("Please enter a valid number.")
            continue


        ifchoice == 1:
            stack_obj.push()
        elifchoice == 2:
            stack_obj.pop()
        elifchoice == 3:
            stack_obj.peek()
```

```
 elif choice == 4:
            stack_obj.display()
        elif choice == 5:
            print("Exiting stack program.")
            break
        else:
            print("Invalid choice! Select numbers between 1 and 5.")

if __name__ == "__main__":
main()
```

Output:

```
================== RESTART: D:/DSA Programs/programs/Week6a.py =========
Enter the size of the stack: 4

----- STACK OPERATIONS -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to push: 5
5 pushed onto the stack.

----- STACK OPERATIONS -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to push: 6
6 pushed onto the stack.

----- STACK OPERATIONS -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 6

----- STACK OPERATIONS -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element is: 5
```

```
----- STACK OPERATIONS -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (Top to Bottom):
| 5 |
-----
```

Explanation:

A**stack** isa lineardata structure that follows the **LIFO (Last In, First Out)** principle. The element inserted last is removed first.

The main stack operations are:

1. **Push** – Adds an element to the top of the stack.
2. **Pop** – Removes the top element from the stack.
3. **Peek** – Displays the top element without removing it.
4. **Display** – Displays all elements in the stack.

Error table:

S.No	Error	Correction
1	IndexError: pop from empty list	Check len(stack) == 0 before pop
2	Stack Overflow	Check len(stack) == MAX_SIZE
3	IndexError: list index out of range	Check whether stack is empty before Peek
4	ValueError	Enter a valid integer

2.Aim.:Write a Python program to implement stack operations such as Push, Pop, Peek, Display using a linked list

PROGRAM

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class StackLinkedList:
    def __init__(self):
        self.top = None # Points to the top node of the stack

    # 1. Push Operation
    def push(self):
        data = int(input("Enter data to push: "))
        new_node = Node(data)

        # New node points to the current top, then becomes the new top
        new_node.next = self.top
        self.top = new_node
        print(f"{data} pushed onto the stack.")

    # 2. Pop Operation
    def pop(self):
        if not self.top:
            print("Stack Underflow! Stack is empty.")
            return

        popped_node = self.top
        self.top = self.top.next # Move top pointer to the next node
        print(f"Popped element: {popped_node.data}")

    # 3. Peek Operation
    def peek(self):
        if not self.top:
            print("Stack is empty!")
            return

        print(f"Top element is: {self.top.data}")
```

```
 # 4. Display Operation
    def display(self):
        if not self.top:
            print("Stack is empty!")
            return

        print("Stack elements (Top to Bottom):")
        temp = self.top
        while temp:
            print(f"| {temp.data} |")
            temp = temp.next
        print("-----")


def main():
    stack_obj = StackLinkedList()

    while True:
        print("\n----- STACK OPERATIONS (LINKED LIST) -----")
        print("1. Push")
        print("2. Pop")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")

        try:
            choice = int(input("Enter your choice: "))
        except ValueError:
            print("Please enter a valid number.")
            continue

        if choice == 1:
            stack_obj.push()
        elif choice == 2:
            stack_obj.pop()
        elif choice == 3:
            stack_obj.peek()
         elif choice == 4:
                stack_obj.display()
          elif choice == 5:
                print("Exiting stack program.")
            break
           else:
            print("Invalid choice! Select numbers between 1 and 5.")
```

Output:

```
= RESTART: C:/Users/Manaswini/AppData/Local/Programs/Python/Python314/Week6b.py
manaswini-cse25102

----- STACK OPERATIONS (LINKED LIST) -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to push: 3
3 pushed onto the stack.

----- STACK OPERATIONS (LINKED LIST) -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to push: 6
6 pushed onto the stack.

----- STACK OPERATIONS (LINKED LIST) -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to push: 8
8 pushed onto the stack.

----- STACK OPERATIONS (LINKED LIST) -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
```

```
----- STACK OPERATIONS (LINKED LIST) -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (Top to Bottom):
| 6 |
| 3 |
-----

----- STACK OPERATIONS (LINKED LIST) -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting stack program.
```

Explanation:

A linked-list stack consists of **nodes**.

A temporary pointer travels through the linked list:

Error table:

S.No	Error	Correction
1	AttributeError	Check self.top is None before accessing data
2	Stack Underflow	Check whether top is None
4	NameError	Use the correct variable/reference

Lab-7

1.Aim: Write a python programto implement queue and its operations such as enqueue, dequeue, peek anddisplay with a menu driven approach using an array.

PROGRAM

```
queue = []
MAX_SIZE = 5

def enqueue():
    iflen(queue) == MAX_SIZE:
        print("Queue Overflow! Queue is full.")
    else:
        item= input("Enter the element to enqueue: ")
        queue.append(item)
        print(item, "inserted into the queue.")

def dequeue():
    iflen(queue) == 0:
        print("Queue Underflow! Queue is empty.")
    else:
        item= queue.pop(0)
        print(item, "deleted from the queue.")

def peek():
    iflen(queue) == 0:
        print("Queue is empty.")
    else:
        print("Front element:", queue[0])

def display():
    iflen(queue) == 0:
        print("Queue is empty.")
    else:
        print("Queue elements:", end=" ")
        foritem in queue:
            print(item, end=" ")
        print()

 #Menu-driven program
 while True:
    print("\n--- QUEUE USING ARRAY ---")
```

```
print("1. Enqueue")
print("2. Dequeue")
print("3. Peek")
print("4. Display")
print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        enqueue()
    elif choice == "2":
        dequeue()
    elif choice == "3":
        peek()
    elif choice == "4":
        display()
    elif choice == "5":
        print("Exiting program...")
        break
    else:
        print("Invalid choice! Please try again.")
```

Output:

```
5. Exit
Enter your choice: 1
Enter the element to enqueue: 2
2 inserted into the queue.

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 4
4 inserted into the queue.

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
2 deleted from the queue.
```

```
--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
```

```
--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements: 4

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting program...
```

Explanation:

1. Queue follows FIFO (First In, First Out), meaning the element inserted first is removed first.
2. Node creation: Each node stores data and a pointer to the next node.
3. Enqueue: Adds a new element at the rear end of the queue.
4. Dequeue: Removes an element from the front end of the queue.
5. Display: Traverses the queue from front to rear and prints all elements.

Error table:

S.No	Error	Rectification
1	Using queue.pop() instead of queue.pop(0) removes the last element instead of the front element.	Use queue.pop(0) for dequeue operation.
2	Not checking the queue size before insertion can cause the queue to exceed its maximum size.	Check if len(queue) == MAX_SIZE before enqueue.
3	Accessing queue[0] when the queue is empty causes an IndexError.	Check if len(queue) == 0 before performing Peek.

2.AIM: Write a Python program to implement a queue Dequeue, Peek,Display with a menu-driven approach. Implement using structures.

Program:

```
# Circular Queue implementation using Array

MAX_SIZE = 5
queue = [None] * MAX_SIZE

front = -1
rear = -1


def enqueue():
    global front, rear

    #Check if queue is full
    if(rear + 1) % MAX_SIZE == front:
        print("Queue Overflow! Queue is full.")
        return

    item = input("Enter the element to enqueue: ")
```

```
 # First element
    if front == -1:
    front = 0
    rear = 0
    else:
        rear = (rear + 1) % MAX_SIZE

 queue[rear] = item
 print(item, "inserted into the queue.")


def dequeue():
    global front, rear

 # Check if queue is empty
    if front == -1:
        print("Queue Underflow! Queue is empty.")
        return

 item = queue[front]
    queue[front] = None

 # If only one element was present
    if front == rear:
        front = -1
        rear = -1
    else:
        front = (front + 1) % MAX_SIZE

 print(item, "deleted from the queue.")


def peek():
    if front == -1:
        print("Queue is empty.")
    else:
        print("Front element:", queue[front])

def display():
    if front == -1:
        print("Queue is empty.")
        return

 print("Queue elements:", end=" ")
```

```
    i = front

    while True:
        print(queue[i], end=" ")

        if i == rear:
            break

        i = (i + 1) % MAX_SIZE

    print()


# Menu-driven program
while True:
    print("\n--- CIRCULAR QUEUE USING ARRAY ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        enqueue()

    elif choice == "2":
        dequeue()

    elif choice == "3":
        peek()

    elif choice == "4":
        display()

    elif choice == "5":
        print("Exiting program...")
        break

    else:
        print("Invalid choice! Please try again.")
```

Output:

```
= RESTART: C:/Users/Manaswini/AppData/Local/Programs/Python/Python314/Week7b.py
manaswini-cse25102

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 2
2 inserted into the queue.

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 4
4 inserted into the queue.

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 6
6 inserted into the queue.

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
2 deleted from the queue.

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 4

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements: 4 6

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting program...
```

Explanation:

A **linked-list queue** consists of dynamically created nodes. Each node contains two parts: Two pointers are maintained:

Front: Points to the first node. **Rear:** Points to the last node.

Error table:

S.No	Error	Rectification
1	After deleting the last node, rear may still point to the deleted node.	If front becomes None, set self.rear = None.
2	Displaying the linked list without moving to the next node can create an infinite loop.	Use current = current.next inside the display loop.

LAB-8

Aim:A) Write a python program to implement a single linked list with the below operations:

program :

```
print("manaswini-cse25102")
class BinaryTree:
    def__init__(self, size):
        self.tree = [None] * size
        self.size = size

    definsert(self, value, index=0):
        ifindex >= self.size:
            print("Tree is full")
            return

        ifself.tree[index] is None:
            self.tree[index] = value
        elifvalue < self.tree[index]:
            self.insert(value, 2 * index + 1)
        else:
            self.insert(value, 2 * index + 2)

    definorder(self, index=0):
        ifindex >= self.size or self.tree[index] is None:
            return

        self.inorder(2 * index + 1)
        print(self.tree[index], end=" ")
        self.inorder(2 * index + 2)

    defpreorder(self, index=0):
        ifindex >= self.size or self.tree[index] is None:
            return

        print(self.tree[index], end=" ")
        self.preorder(2 * index + 1)
        self.preorder(2 * index + 2)

    defpostorder(self, index=0):
        ifind76e x| P >ag=e self.size or self.tree[index] is None:
            return
```

```
self.postorder(2 * index + 1)
  self.postorder(2 * index + 2)
 print(self.tree[index], end=" ")


    bt = BinaryTree(10)

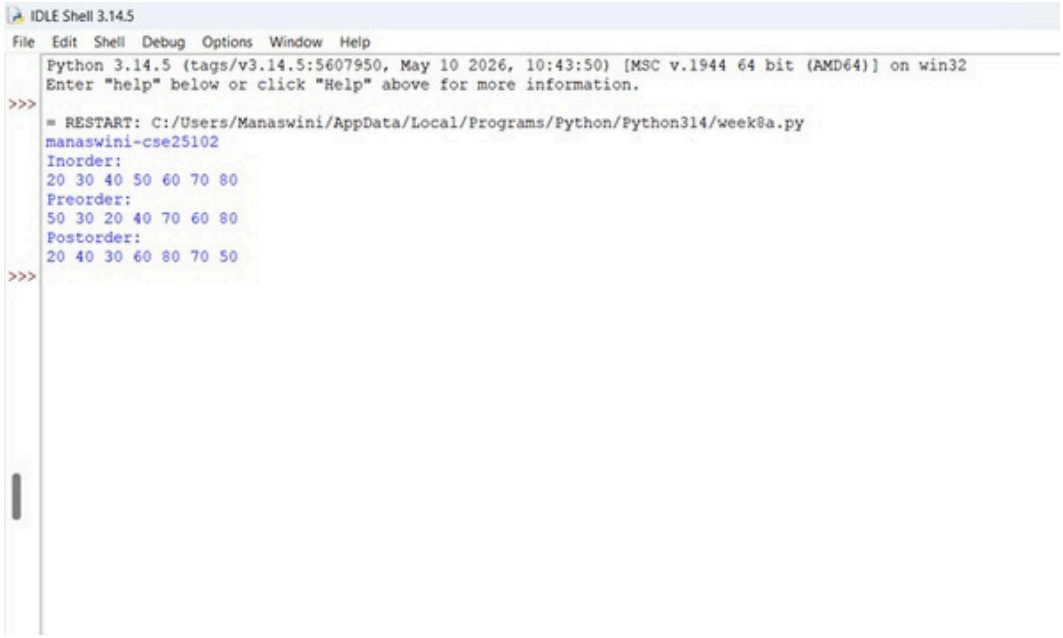
        bt.insert(50)
        bt.insert(30)
        bt.insert(70)
        bt.insert(20)
        bt.insert(40)
        bt.insert(60)
        bt.insert(80)

      print("Inorder:")
        bt.inorder()

    print("\nPreorder:")
        bt.preorder()

    print("\nPostorder:")
        bt.postorder()
```

Output:



Explanation:

1. Class: BinaryTree defines the binary tree structure.
2. Initialization: __init__() creates a list of the given size filled with None.
3. Insertion: insert() adds elements to the binary search tree.
4. Left child: 2 * index + 1 represents the left child.
5. Right child: 2 * index + 2 represents the right child.
6. Comparison: Smaller values go to the left, while larger or equal values go to the right.
7. Inorder traversal: Visits Left → Root → Right and displays values in ascending order.
8. Preorder traversal: Visits Root → Left → Right.
9. Postorder traversal: Visits Left → Right → Root.
10. Tree creation: bt = BinaryTree(10) creates a tree with a maximum array size of 10.

Error	Explanation
IndexError	Occurs if the index goes outside the tree size; checked
Tree is full	Printed when the insertion index exceeds the available
None value	Indicates an empty position in the tree.
Incorrect traversal	Inorder, preorder, and postorder must follow their
Duplicate values	Duplicate values are inserted into the right subtree

Aim:B) Write a python program to implement a binary tree with its operations using linked list.

program :

```
print("manaswini-cse25102")
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BinaryTree:
    def __init__(self):
        self.root = None

    def insert(self, value):
        new_node = Node(value)

        if self.root is None:
            self.root = new_node
            return

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.left is None:
                current.left = new_node
                return
            else:
                queue.append(current.left)

            if current.right is None:
                current.right = new_node
                return
            else:
                queue.append(current.right)

    def inorder(self, node):
        if node is not None:
            self.inorder(node.left)
            print(node.data,end=" ")
            self.inorder(node.right)
```

```
defpreorder(self, node):
        ifnode is not None:
    print(node.data, end=" ")
     self.preorder(node.left)
    self.preorder(node.right)

defpostorder(self, node):
      ifnode is not None:
     self.postorder(node.left)
    self.postorder(node.right)
    print(node.data, end=" ")

    bt = BinaryTree()

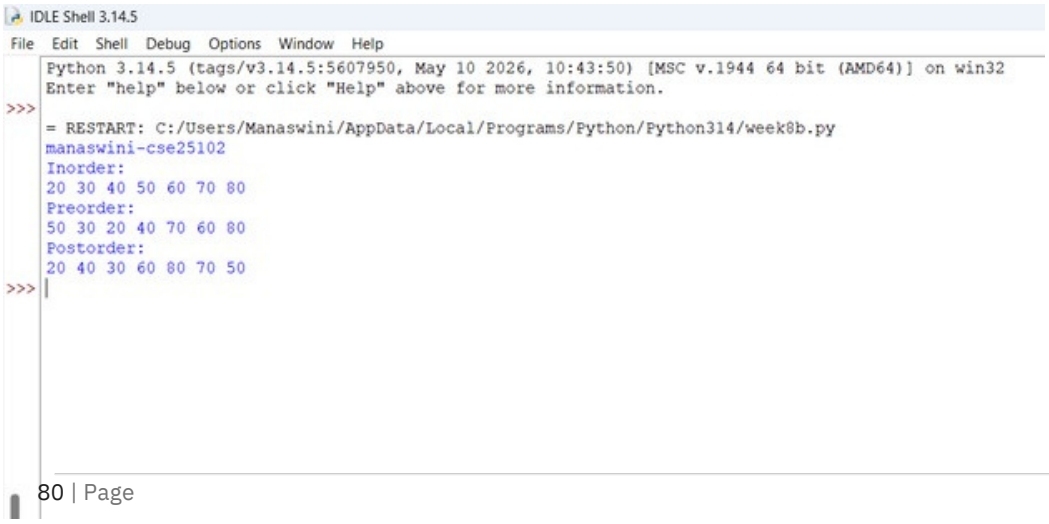
        bt.insert(50)
        bt.insert(30)
        bt.insert(70)
        bt.insert(20)
        bt.insert(40)
        bt.insert(60)
        bt.insert(80)

     print("Inorder:")
    bt.inorder(bt.root)

  print("\nPreorder:")
  bt.preorder(bt.root)

 print("\nPostorder:")
 bt.postorder(bt.root)
```

Output:



Explanation:

1. Class Node: Creates a node containing data, a left child, and a right child.

2. Initialization: __init__() initializes the node data and sets left and right to None.

3. Class BinaryTree: Defines the binary tree structure.

4. Root: self.root = None initializes an empty tree.

5. Insertion: The insert() method creates a new node and inserts it into the tree.

6. Queue: A queue is used to find the first empty position level by level.

7. Left and Right Children: If the left child is empty, the new node is inserted there; otherwise, the right child is checked.

8. Inorder Traversal: Visits Left → Root → Right.

9. Preorder Traversal: Visits Root → Left → Right.

10. Postorder Traversal: Visits Left → Right → Root.

Error	Explanation
Root not created	The tree cannot work properly if the root node is
Node not inserted	A new node may not be added if the insertion logic
Wrong traversal order	The output becomes incorrect if nodes are
Left or right child missing	A node may not be connected properly to its
Incorrect output	The traversal output may differ if the tree structure

LAB-9

1.Aim: write a python programtoimplementa binary search tree with its operations and traversals:

program :

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root


def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)

def preorder(root):
    if root is None:
        return
    print(root.data, end=" ")
    preorder(root.left)
    preorder(root.right)

def postorder(root):
    if root is None:
        return
    postorder(root.left)
    postorder(root.right)
    print(root.data, end=" ")
```

```
def find_min(root):
    if root is None:
        return None
    while root.left is not None:
        root = root.left
    return root

def find_max(root):
    if root is None:
        return None
    while root.right is not None:
        root = root.right
    return root

def search(root, data):
    if root is None:
        return None
    if data < root.data:
        return search(root.left, data)
    elif data > root.data:
        return search(root.right, data)
    else:
        return root

def count_nodes(root):
    if root is None:
        return 0
    return count_nodes(root.left) + count_nodes(root.right) + 1


def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)

def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right)
+ 1
```

```
def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
 return max(left_height, right_height) + 1


def delete(root, x):
    if root is None:
        print("Element not found.")
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        # Case 1: Node has no children
        if root.left is None and root.right is None:
            root = None
        # Case 2: Node has only right child
        elif root.left is None:
        root = root.right
        # Case 3: Node has only left child
        elif root.right is None:
            root = root.left
        # Case 4: Node has two children
        else:
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root

def print_level(root, level):
    if root is None:
        return
    if level == 0:
        print(root.data, end=" ")
    else:
        print_level(root.left, level - 1)
        print_level(root.right, level - 1)
```

```
def level_order(root):
    h = height(root)
    for level in range(h):
        print_level(root, level)
# ========================================================
# MAIN PROGRAM
# ========================================================
root = None
while True:
    print("\n\n******** BINARY SEARCH TREE ********")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
 print("5. FIND MINIMUM")
print("6. FIND MAXIMUM")
    print("7. SEARCH")
    print("8. DELETE")
    print("9. COUNT NODES")
    print("10. COUNT LEAVES")
    print("11. COUNT INTERNAL NODES")
    print("12. HEIGHT")
    print("13. LEVEL ORDER TRAVERSAL")
    print("14. EXIT")

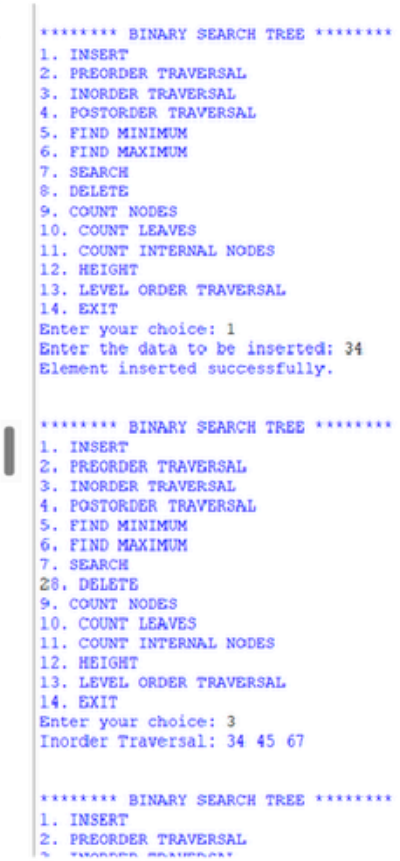
    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
        print("Element inserted successfully.")
    elif choice == 2:
        print("Preorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
    preorder(root)
    print()
    elif choice == 3:
        print("Inorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            inorder(root)
            print()
```

```
elifchoice == 4:
    print("Postorder Traversal:", end=" ")
    ifroot is None:
        print("Tree is empty.")
    else:
  postorder(root)
  print()
  elifchoice == 5:
    node = find_min(root)
    ifnode is None:
        print("Tree is empty.")
    else:
        print("Minimum element:", node.data)
  elifchoice == 6:
    node = find_max(root)
    ifnode is None:
        print("Tree is empty.")
    else:
        print("Maximum element:", node.data)
  elifchoice == 7:
    data = int(input("Enter the data to be searched: "))
    node = search(root, data)
    ifnode is None:
        print("Element", data, "is not found.")
    else:
        print("Element", data, "is found.")
  elifchoice == 8:
    data = int(input("Enter the data to be deleted: "))
    root = delete(root, data)
  elifchoice == 9:
    print("Number of nodes:", count_nodes(root))
  elifchoice == 10:
    print("Number of leaf nodes:", count_leaves(root))
  elifchoice == 11:
    print("Number of internal nodes:", count_internal(root))
  elifchoice == 12:
    print("Height of BST:", height(root))
  elifchoice == 13:
    print("Level Order Traversal:", end=" ")
    ifroot is None:
        print("Tree is empty.")
```

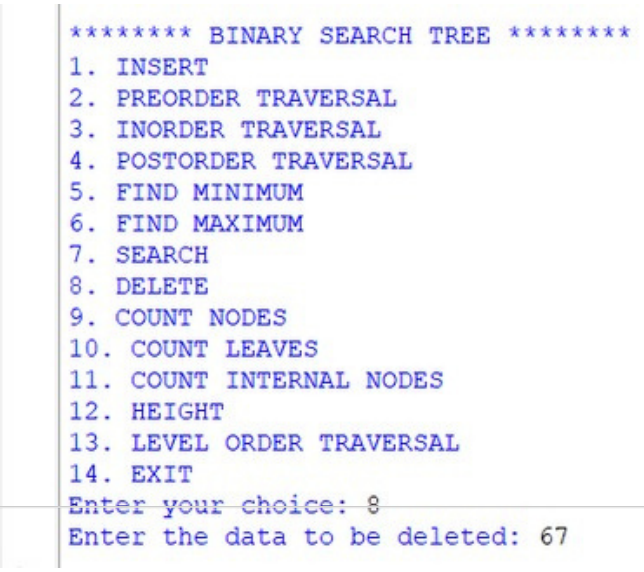
```
else:
        level _order(root)
        print()
  elifchoice == 14:
    print("Program terminated.")
    break
  else:
    print("Invalid choice. Please try again.")
```

Output:









```
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 12
Height of BST: 1
```

Explanation:

1. Node Class: Creates a node containing data, a left child, and a right child. Initially, both children are None.
2. Insert: Adds a new value to the BST. If the value is smaller than the current node, it goes left; if larger, it goes right. Duplicate values are not allowed.
3. Preorder Traversal: Visits nodes in the order Root → Left → Right.
4. Inorder Traversal: Visits nodes in the order Left → Root → Right. In a BST, this displays values in ascending order.
5. Postorder Traversal: Visits nodes in the order Left → Right → Root.
6. Find Minimum: Finds the smallest value by moving repeatedly to the leftmost node.
7. Find Maximum: Finds the largest value by moving repeatedly to the rightmost node.

LAB-10

AIM: write a python programtoimplementanbinaryAVL tree with its operations

Program:

```
print("Manaswini-CSE25102")
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1


def height(root):
    if root is None:
        return 0
    return root.height


 def balance(root):
     if root is None:
        return 0
    return height(root.left) - height(root.right)


def right_rotate(y):
    x = y.left
    t = x.right

    x.right = y
    y.left = t

 y.height = 1 + max(height(y.left), height(y.right))
x.height = 1 + max(height(x.left), height(x.right))

    return x
```

```
def left_rotate(x):
    y = x.right
    t = y.left

    y.left = x
    x.right = t

 x.height = 1 + max(height(x.left), height(x.right))
y.height = 1 + max(height(y.left), height(y.right))

    return y

def insert(root, data):
    if root is None:
        return Node(data)

    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)
    else:
        return root

 root.height = 1 + max(height(root.left), height(root.right))
    b = balance(root)

    # LL case
    if b > 1 and data < root.left.data:
        return right_rotate(root)

    # RR case
    if b < -1 and data > root.right.data:
        return left_rotate(root)

    # LR case
    if b > 1 and data > root.left.data:
        root.left = left_rotate(root.left)
        return right_rotate(root)

    # RL case
    if b < -1 and data < root.right.data:
        root.right = right_rotate(root.right)
        return left_rotate(root)
```

---

r9e0t|u Pranger root

```
def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)


def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)


root = None

while True:
    print("\n\n1. Insert")
    print("2. Inorder Traversal")
    print("3. Preorder Traversal")
    print("4. Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        x = int(input("Enter value: "))
        root = insert(root, x)
    elif ch == 2:
        inorder(root)
    elif ch == 3:
        preorder(root)
    elif ch == 4:
        break
    else:
        print("Invalid choice")
```

Output:

```
================== RESTART: D:/DSA Programs/programs/Week10.py ==================
Manaswini-CSE25102

1. Insert
2. Inorder Traversal
3. Preorder Traversal
4. Exit
Enter choice: 1
Enter value: 2

1. Insert
2. Inorder Traversal
3. Preorder Traversal
4. Exit
Enter choice: 2
2

1. Insert
2. Inorder Traversal
3. Preorder Traversal
4. Exit
Enter choice: 1
Enter value: 4

1. Insert
2. Inorder Traversal
3. Preorder Traversal
4. Exit
Enter choice: 3
2 4

1. Insert
2. Inorder Traversal
3. Preorder Traversal
4. Exit
Enter choice: 4
```

Explanation:

AVLTree: Aself-balancing Binary Search Tree in which the balance factor of every node is -1, 0, or +1.

Insertion and Rotations: Insertion adds a node according to BST rules. Rotations (LL, RR, LR, RL) maintain the balance of the tree.

Traversal Operations: Inorder visits Left → Root → Right, Preorder visits Root → Left → Right, and Postorder visits Left → Right → Root.

Find Minimum and Maximum: The minimum value is found at the leftmost node, and the maximum value is found at the rightmost node.

Search Operation: Searches for a given value by comparing it with the current node and moving left or right accordingly.

Delete Operation: Removes the specified node, updates the heights, and performs rotations when necessary to maintain AVL tree balance.

Error Table:

S.No.	Error Type	Error / Mistake	Correction
1	Logical Error	Incorrect balance factor may cause the AVL tree to become unbalanced.	Calculate the balance factor as height(root.left)-height(root.right).
2	Logical Error	Incorrect rotation can violate the AVL tree's balancing rules.	Use the correct LL, RR, LR, or RL rotation.
3	Runtime Error	Accessing a node that is None may cause an error.	Check whether the node exists before accessing its attributes.
4	Logical Error	Deleting a node without updating heights can make the AVL tree unbalanced.	Update node heights and rebalance the tree after deletion.
5	Logical Error	Searching for a value that does not exist may return None.	Display Not Found when the search returns None.
6	Logical Error	Entering a non-integer value for the menu choice may cause a ValueError.	Enter a valid integer for the menu choice.